PSI-2026 Sep ASDE GGN

StoreConnect – Omnichannel Retail Platform Capstone Project (CPS)

Progressive Full-Stack Engineering Problem Statement

CPS Delivery Model: The CPS is developed progressively during the 20-day learning phase and demonstrated on Day 21. The objective is a focused, working full-stack MVP—not a production-ready retail platform.
Scope Boundary: Docker, Kubernetes and AWS are part of the learning TOC but are explicitly outside the CPS implementation scope. Python is also outside the StoreConnect CPS scope because it is taught after the CPS demo but CPS will be refactored in Python from Day 22 – Day 25.

1. CPS Overview

Project Name: StoreConnect – Omnichannel Retail Platform

Industry: Retail / Omnichannel Commerce

StoreConnect is a digital retail platform for a growing retailer that operates physical stores alongside an online shopping channel. The platform connects the customer shopping experience with store-level product availability, inventory operations and order fulfillment.

CPS Design Principle

Associates will evolve the same product progressively as they learn each technology. The CPS is not a separate end-of-training assignment; every learning module should result in a tangible improvement to StoreConnect.

Physical-store simulation

Physical stores are represented digitally as store/location records and inventory in the application. Associates are not expected to integrate with real stores, POS hardware, barcode scanners, payment systems or logistics providers.

2. Business Problem Statement

A growing retail organization currently manages product catalog, store inventory and customer orders through multiple disconnected processes. Customers may see a product as available online even when the nearest store does not have sufficient stock. Store teams have limited visibility into incoming online orders, inventory commitments and fulfillment status.

The organization wants to build StoreConnect, a unified digital retail platform that enables customers to discover products, view availability, place orders and track fulfillment, while giving retail administrators and store operators better visibility and control over catalog, inventory and orders.

Business Challenge	StoreConnect Response
Disconnected product and inventory information	Provide a consistent product and availability view.
Unclear store-level stock availability	Maintain inventory by store/location and expose relevant availability.
Manual order handling	Create a digital order lifecycle with clear status transitions.
Limited store-team visibility	Provide operational views for inventory and order fulfillment.
Limited event visibility	Use a selected asynchronous event for an order/inventory business event.

3. CPS Objectives

- Apply modern Java and object-oriented programming concepts to a realistic retail business problem.
- Translate business requirements into focused user stories, acceptance criteria, domain objects and working features.
- Practice Git workflow, meaningful commits, branching and code review/clean-code practices.
- Design and implement a focused product, store, inventory, cart and order data model.
- Develop REST APIs using Spring Boot and apply layered and microservice-oriented application principles.
- Build a customer-facing React + TypeScript interface with appropriate shared state using Redux Toolkit.
- Implement a focused event-driven use case using Kafka.
- Apply unit testing and selected end-to-end testing to critical business journeys.
- Demonstrate an end-to-end flow across frontend, APIs, services, database and event processing.

4. User Roles

Role	Primary Responsibilities
Customer	Register/login, browse/search products, view product/store availability, manage cart, place orders and track status.
Retail Administrator	Manage products/categories, monitor inventory and orders, and view basic operational information.
Store Operator	View store inventory, update stock, review incoming orders and update fulfillment status.
System	Validate inventory, reserve/release stock, update order states and publish selected business events.

5. Functional Scope

5.1 Customer Experience

- Customer registration/login with application-level authentication.
- Browse and search products by name/category.
- View product details and store-level availability.
- Add, update and remove items from cart.
- Place an order using available inventory.
- View order confirmation, order history and current order status.

5.2 Product & Catalog Management

- Create, update and deactivate products.
- Manage basic categories and product attributes.
- Maintain price and product availability information.

5.3 Store & Inventory Management

- Maintain inventory quantity by store/location.
- Update stock levels.
- Display product availability based on maintained inventory.
- Reserve inventory when an order is confirmed.
- Release/restore reserved stock when an order is cancelled, where applicable.

5.4 Order & Fulfillment Management

- Create an order from the customer's cart.
- Validate stock before confirmation.
- Associate an order with a fulfillment/store location.
- Maintain a simplified lifecycle: CREATED → CONFIRMED → PROCESSING → READY → DELIVERED or CANCELLED.
- Allow store operators to update fulfillment status.
- Expose the current order status to the customer.

Scope control: The above represents the target MVP. Associates should prioritize Must-Have stories first. Low-stock reporting, advanced admin capabilities and additional integrations are optional only if the core end-to-end flow is stable.

6. Key Business Rules

Rule	Expected Behaviour
Inventory validation	An order should not be confirmed when required stock is unavailable.
Stock reservation	Once an order is confirmed, the required inventory should be reserved/reduced according to the chosen implementation approach.
Cancellation	Cancelled orders should release/restore reserved stock where applicable.
Availability	Customer-facing availability should be derived from maintained store inventory.
Order lifecycle	Order status transitions should follow defined rules and should not allow invalid transitions.
Fulfillment location	An order should be associated with a valid store/location for fulfillment.
Event publishing	A selected business event should be published for an important order/inventory change.

7. Initial User Stories

Epic	Sample User Story	Priority
Customer Access	As a customer, I want to register and log in so that I can securely access my account.	Must

Product Discovery	As a customer, I want to search/filter products so that I can find relevant products.	Must
Availability	As a customer, I want to see product/store availability so that I know whether I can order.	Must
Cart	As a customer, I want to add, update and remove products from my cart.	Must
Order	As a customer, I want to place an order using available inventory.	Must
Tracking	As a customer, I want to view my order history and current order status.	Must
Catalog Admin	As an admin, I want to create/update products so that the catalog remains current.	Must
Inventory	As a store operator, I want to update store stock so that product availability remains accurate.	Must
Fulfillment	As a store operator, I want to view incoming orders and update fulfillment status.	Must
Store Availability	As a customer, I want to see availability at a relevant store/location.	Should
Low Stock	As an admin, I want to identify low-stock products so that replenishment can be planned.	Could

8. Non-Functional Requirements

- Security: protected customer/admin functions should require authenticated access.
- Maintainability: apply separation of concerns, meaningful naming and reusable components.
- Reliability: validate inputs and handle expected business/technical exceptions gracefully.
- Data consistency: inventory and order updates should follow clear business rules.
- Observability: provide useful application logs for important order/inventory operations.
- Testability: critical business logic and selected API/UI journeys should be automated.
- Usability: customer-facing flows should provide clear navigation, validation feedback and meaningful errors.
- Scalability awareness: service boundaries and event-driven processing should consider future growth without requiring production-scale infrastructure.

9. Technology & Learning Alignment

Learning Area	StoreConnect Application
Maven / Git / CI concepts	Project structure, dependency management, repository, branching and build workflow.
Java 25	Domain models, business rules, services and modern Java language features.
Collections / Streams / Concurrency	Catalog, inventory, cart/order operations and selected backend processing.
PostgreSQL / JDBC	Core customers, products, stores, inventory, carts and orders persistence.
MongoDB / NoSQL	One focused secondary use case such as activity/audit data, if justified.
Enterprise Design Principles	Layered architecture, DTOs, separation of concerns and service boundaries.
Spring Core / IoC / MVC / Spring Boot	Dependency injection, backend structure and application services.
REST / OpenAPI / Swagger	Product, inventory, store and order APIs with documented contracts.
Microservices / Spring Cloud	Focused Catalog, Inventory and Order service boundaries; inter-service communication.
JUnit / Mockito / TDD / Code Quality	Business-rule and service-layer tests; quality review.
Kafka	One meaningful order/inventory business event using Spring Boot integration.
JavaScript / Node.js	Frontend tooling and basic JavaScript/Node ecosystem understanding.
TypeScript	Typed frontend models, API contracts and application code.
React	Customer/admin interfaces and application flow.
Redux Toolkit	Shared state for cart/product/order state where justified.
Selenium / Cucumber	One or two critical end-to-end customer journeys.

10. 20-Day Progressive CPS Integration Plan

The CPS is scoped to the first 20 learning days. Docker, Kubernetes and AWS are excluded from CPS implementation, and Day 21 is reserved for final CPS consolidation and demonstration. The objective is to reach a working full-stack MVP through incremental development.

Day(s)	Learning Topic(s)	CPS Application
Days 1–2	OOAD/UML, Maven/Git, Java 25	Understand the retail problem; define actors/use cases; create initial domain model and repository; implement Product, Store, Customer and Inventory domain objects.
Day 3	Collections, Streams & Concurrency	Implement core catalog/inventory business operations and business-rule logic using collections, streams and appropriate Java constructs.
Day 4	PostgreSQL, SQL & JDBC	Design the core relational model and persist products, stores, inventory, customers and orders using PostgreSQL/JDBC.
Day 5	MongoDB & NoSQL	Implement one small, justified secondary use case such as activity/audit data; avoid duplicating the relational model.
Day 6	CI/CD with Jenkins + CPS Work 1	Create the first working CPS baseline, integrate repository/build workflow, review domain/data implementation and identify backlog for the backend.
Day 7	Java APIs & Enterprise Design	Refine the codebase into maintainable layers; apply DTOs, validation, design principles and date/time/resource-handling where useful.
Day 8	Spring Core / IoC / MVC	Convert the backend into a Spring application with dependency injection, controller/service/repository separation and configuration.
Day 9	Spring Boot / REST / OpenAPI	Expose core catalog, inventory and order APIs; add validation, exception handling, logging and API

		documentation.
Day 10	Microservices / Spring Cloud / Observability	Establish focused Catalog, Inventory and Order service boundaries; implement selected inter-service communication and resilience/observability concepts.
Day 11	Quality Engineering	Add unit tests for critical business rules and service dependencies; perform quality review and address defects.
Day 12	Kafka + CPS Work 2	Complete core backend integration; add one meaningful order/inventory event and review service/API integration.
Day 13	JavaScript & Node.js	Prepare the frontend ecosystem; use modern JavaScript, modules, async programming and npm/Node tooling as required by the frontend.
Day 14	TypeScript	Create typed models, API contracts and frontend service functions; establish the typed foundation for the CPS UI.
Day 15	React Fundamentals	Build the initial customer-facing interface: product listing, product details, basic cart and reusable components.
Day 16	Modern React / API Integration	Connect React to Spring APIs; implement loading/error states, routing, forms and core customer journey.
Day 17	Redux Toolkit	Introduce shared state for cart/product/order flows where justified; stabilize frontend application structure.
Day 18	BDD/Selenium/Cucumber + CPS Work 3	Complete frontend-backend integration and automate one or two critical journeys such as login, product discovery, cart or order placement.
Days 19–20	Integration & Stabilization	Complete the end-to-end MVP, fix integration defects, strengthen tests, logging, documentation and business-flow consistency. No Docker/Kubernetes/AWS implementation is required.
Day 21	CPS Finalization & Demo	Final walkthrough: business problem, architecture, customer flow, store/inventory flow, APIs, event flow, testing evidence, known limitations and feedback.

11. Suggested CPS Checkpoints

Checkpoint	Expected Demonstration	Evidence
1 – Foundation	Problem understanding, actors, use cases, domain model, repository and initial Java solution.	UML + repository + domain code
2 – Backend & Data	PostgreSQL persistence, Spring Boot structure, core REST APIs and business rules.	ERD + DB scripts + APIs + Swagger/OpenAPI
3 – Full-Stack Integration	React/TypeScript/Redux connected to backend; product, cart and order flow working.	Working UI + API integration + shared state
4 – Final Engineering Review	Kafka event, automated tests, critical E2E journey, logs, README and end-to-end walkthrough.	Tests + event flow + UI demo + documentation
5 – Final Demo (Day 21)	Demonstrate the complete MVP and explain architecture, business flow, engineering choices and known limitations.	End-to-end demo + architecture walkthrough

12. Expected CPS Deliverables

- Problem understanding, assumptions, scope and prioritized backlog.
- Actor identification, user stories and acceptance criteria.
- Use-case/UML artefacts and high-level solution design.
- Database design and relevant PostgreSQL schema/scripts.
- Focused Java/Spring Boot microservice-based backend implementation.
- REST APIs with OpenAPI/Swagger documentation.
- React + TypeScript customer-facing interface with Redux Toolkit where justified.
- One selected Kafka event-driven use case.
- Automated unit tests and selected end-to-end test scenarios.
- Git repository with meaningful commits and branching history.
- Logging, validation and exception-handling implementation.
- README with setup instructions, architecture overview, API information, demo flow and known limitations.
- Final CPS demonstration covering the key customer-to-order-to-fulfillment journey.

13. CPS Scope Exclusions

- Cloud deployment of the CPS is excluded.
- Docker implementation/containerization, Kubernetes implementation/orchestration and AWS implementation/deployment is excluded from the CPS, even though Docker, Kuberenetes, AWS is covered in the learning TOC.
- Python is excluded from the StoreConnect CPS; the Python learning block follows the CPS demonstration.
- Real payment gateway integration is excluded; payment/order confirmation may be simulated.
- Real logistics/shipping-provider integration is excluded.
- Real-time GPS/delivery-partner tracking is excluded.

- Real POS hardware, barcode scanners, physical store systems and external enterprise integrations are excluded.
- Production-grade infrastructure, high availability and enterprise-scale operations are outside the CPS scope.
- Advanced analytics, complex reporting and additional low-stock automation are optional and should not displace the core end-to-end flow.
- Feature scope must be prioritized into Must Have / Should Have / Could Have; the core MVP takes precedence over technology checklist completion.

Definition of Done for the Learning-Phase CPS: A group should be able to demonstrate one complete, working business journey: customer discovers a product → checks store availability → adds it to cart → places an order → inventory is validated/reserved → the order is associated with a store → store fulfillment status is updated → customer sees the updated order status. The implementation should be supported by Spring Boot APIs, React/TypeScript UI, PostgreSQL persistence, appropriate service boundaries, automated tests and one selected Kafka event.